Python-Data-Structure

What is a Python Data Structure?

- A Python data structure is a way of organizing and storing data in a computer so it can be accessed and modified efficiently.
- In Python, data structures help you manage collections of data such as numbers, names, records, or objects.
- Think of a data structure as a container that holds data in a specific format.

Built-in Python Data Structures

- List
- Tuple
- Set
- Dictionary

List

- A list is a built-in data structure used to store multiple items in a single variable.
- Think of a list as a container that keeps items in order.
- Characteristics of a List
- ■ Ordered – Items have a fixed position (index starts at 0)■ Mutable – You can change, add, or remove items■ Allows Duplicates – Same value can appear multiple times■ Can Store Different Data Types

Slide 5

- Creating a List
- Lists are created using square brackets [ ].
- numbers = [10, 20, 30, 40]
- print(numbers)
- Accessing List Elements (Indexing)
- Each item has an index starting from 0.
- fruits = ["apple", "banana", "cherry"]
- print(fruits[0]) # apple
- print(fruits[1]) # banana
- You can also use negative indexing:
- print(fruits[-1]) # cherry

Slide 6

- Modifying a List (Mutable)
- You can change values in a list.
- fruits = ["apple", "banana", "cherry"]
- fruits[1] = "mango"
- print(fruits)
- # ['apple', 'mango', 'cherry']

Slide 7

- Adding Items
- ➤ append() – adds one item at the end
- fruits.append("orange")
- ➤ insert() – adds item at specific position
- fruits.insert(1, "grapes")
- Removing Items
- ➤ remove() – removes specific value
- fruits.remove("apple")
- ➤ pop() – removes by index
- fruits.pop(0)

Slide 8

- Looping Through a List
- students = ["Ana", "Ben", "Carlo"]
- for student in students:
- print(student)

Real-Life Examples using list in python

- Example 1: Storing Student Scores
- scores = [85, 90, 78, 92]
- print("Highest score:", max(scores))
- Example 2: Shopping Cart
- cart = ["milk", "bread", "eggs"]
- cart.append("butter")
- print(cart)

Slide 10

- Summary
- A list in Python is:
- A collection of items
- Ordered
- Changeable (mutable)

- Allows duplicate values

TUPLE

- A tuple is a built-in data structure used to store multiple items in one variable — similar to a list, but it cannot be changed after creation.
- Think of a tuple as a sealed container. Once created, its contents cannot be modified.

Slide 12

- Characteristics of a Tuple
- ■ Ordered – Items keep their position■ Immutable – Cannot be changed after creation■ Allows Duplicates■ Can store different data types

Slide 13

- Creating a Tuple
- Tuples use parentheses ( ).\
- numbers = (10, 20, 30)
- print(numbers)
- Accessing Tuple Elements
- Indexing starts at 0.
- fruits = ("apple", "banana", "cherry")
- print(fruits[0]) # apple
- print(fruits[1]) # banana
- Negative indexing works too:
- print(fruits[-1]) # cherry

Slide 14

- Tuple is Immutable
- You cannot change values:
- fruits = ("apple", "banana", "cherry")
- fruits[1] = "mango" # ■ Error
- This will produce an error because tuples cannot be modified.

Slide 15

- Looping Through a Tuple
- colors = ("red", "blue", "green")
- for color in colors:
- print(color)
- When to Use a Tuple
- Use a tuple when:
- Data should NOT change
- You want to protect data from modification
- You are storing fixed values

Slide 16

- Example 1: GPS Coordinates (Fixed Data)
- location = (14.5995, 120.9842)
- print("Latitude:", location[0])
- Coordinates should not change — so tuple is ideal.
- Example 2: Days of the Week
- days = ("Monday", "Tuesday", "Wednesday",
- "Thursday", "Friday", "Saturday", "Sunday")
- print(days[5]) # Saturday
- Days are constant — perfect for a tuple.

Slide 17

- Summary
- A tuple is:
- Ordered
- Immutable
- Allows duplicates
- Written using ( )

Set in Python

- A set is a built-in data structure used to store multiple unique items in a single variable.
- Think of a set like a collection of unique objects — duplicates are automatically removed.
- Characteristics of a Set
- ■ Unordered – Items do not have a fixed position■ No Duplicates Allowed■ Mutable – You can add or remove items■ No indexing (because unordered)

Slide 19

- Creating a Set
- Sets are created using curly braces { }.
- numbers = {1, 2, 3, 4}
- print(numbers)
- If duplicates are included, Python removes them:

- values = {1, 2, 2, 3, 3, 3}
- print(values) # {1, 2, 3}

Slide 20

- Accessing Set Elements
- You cannot use indexing like set[0] because sets are unordered.
- But you can loop through a set:
- colors = {"red", "blue", "green"}
- for color in colors:
- print(color)

Slide 21

- Adding Items to a Set
- ➤ add() – Adds one item
- fruits = {"apple", "banana"}
- fruits.add("orange")
- print(fruits)
- Removing Items from a Set
- ➤ remove() – Removes specific item
- fruits.remove("banana")
- print(fruits)

Slide 22

- Set Operations (Very Powerful Feature)
- Sets are commonly used in mathematics.
- ➤ Union (combine sets)
- A = {1, 2, 3}
- B = {3, 4, 5}
- print(A | B) # {1, 2, 3, 4, 5}

Slide 23

- ➤ Intersection (common elements)
- print(A & B) # {3}
- ➤ Difference (items in A not in B)
- print(A - B) # {1, 2}

Real-Life Examples

- Example 1: Unique Student IDs
- student_ids = {1001, 1002, 1003, 1002}
- print(student_ids)
- Duplicate IDs are automatically removed.

Slide 25

- Example 2: Common Interests Between Two People
- person1 = {"music", "sports", "reading"}
- person2 = {"sports", "travel", "music"}
- common = person1 & person2
- print(common)

Slide 26

- When to Use a Set
- Use a set when:
- You need unique values
- You need to perform mathematical operations
- Order does not matter

Slide 27

- Summary
- A set in Python is:
- Unordered
- Mutable
- Does NOT allow duplicates
- Great for mathematical operations

Dictionary in Python

- A dictionary is a built-in data structure used to store data in key–value pairs.
- Think of a dictionary like a real-world dictionary:
- The word is the key
- The definition is the value

Slide 29

- Characteristics of a Dictionary
- ■ Stores data in key : value format■ Ordered (in modern Python versions)■ Mutable (can change, add, remove items)■ Keys must be unique■ No duplicate keys allowed

Slide 30

- Creating a Dictionary

- Dictionaries use curly braces { }.
- student = {
- "name": "Ana",
- "age": 20,
- "course": "BSIT"
- }
- print(student)

Slide 31

- Accessing Values
- You access values using their key.
- print(student["name"]) # Ana
- print(student["age"]) # 20
- You can also use .get():
- print(student.get("course"))

Slide 32

- Modifying a Dictionary
- You can change a value:
- student["age"] = 21
- print(student)
- Adding New Items
- student["year"] = "2nd Year"
- print(student)

Slide 33

- Removing Items
- student.pop("age")
- print(student)
- ➤ del
- del student["course"]

Slide 34

- Looping Through a Dictionary
- ➤ Loop through keys
- for key in student:
- print(key)
- ➤ Loop through both key and value
- for key, value in student.items():
- print(key, ":", value)

Real-Life Examples

- Example 1: Student Record
- student = {
- "id": 2024001,
- "name": "Ben Cruz",
- "course": "BSIT",
- "gpa": 1.75
- }
- print("Student Name:", student["name"])

Slide 36

- Example 2: Product Information
- product = {
- "name": "Laptop",
- "price": 35000,
- "stock": 10
- }
- print("Price:", product["price"])

Slide 37

- When to Use a Dictionary
- Use a dictionary when:
- You need to store related data
- You want to access data using a unique key
- You need fast lookups

Slide 38

- Summary
- A dictionary in Python:
- Stores data in key–value pairs
- Is mutable
- Has unique keys
- Is useful for structured data